

Adroit Technologies

Protocol Driver

Name	Sigfox Protocol Driver
DLL	D_SIGFOX



COPYRIGHT

Copyright © 2022 Advanced Worx 112 (Pty) Ltd. All rights reserved.

Information in this document is subject to change without notice. The software described in this document is furnished under a license agreement or nondisclosure agreement. The software may be used or copied only in accordance with the terms of those agreements. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or any means electronic or mechanical, including photocopying and recording for any purpose other than the purchaser's personal use without the written permission of Advanced Worx 112 (Pty) Ltd.

Advanced Worx 112 (Pty) Ltd
20 Waterford Office Park
189 Witkoppen Road (c/o Waterford Drive)
Fourways
South Africa

<https://adroittech.co.za/>

Contents

1. Overview	4
2. Wiring	5
3. Device Configuration	6
3.1. Part 1	6
3.1.1. Setting up Devices on the Sigfox Cloud	6
3.1.2. Sigfox Backend Hierarchal Structure	7
3.1.3. Creating a new Device Type	8
3.1.4. Creating a new Device.....	9
3.1.5. Setting up the Sigfox Callback.....	9
3.1.6. Setup Validation	10
3.2. Part 2.....	11
3.2.1. Setting Up the Adroit Sigfox Driver	11
3.3. Configuration Dialog	13
3.3.1. Driver Details	14
3.3.2. Primary Connection Parameters	14
3.3.3. REST API	14
3.3.4. HTTP Server.....	14
3.3.5. Script.....	14
3.3.6. TimeOut	14
4. Supported Addresses	15
4.1. Example	16
5. Protocol Definition and Considerations	17
5.1. Final Verification	17
6. General Notes and Troubleshooting	18
6.1. Contact Details.....	18
6.2. Adroit Technologies Knowledge Base.....	18
6.3. Troubleshooting	18
7. Revisions.....	21
7.1. Document Revision.....	21
7.2. DLL Revision	21

1. Overview

Sigfox is a Low Power Wide Area Network (LPWAN) technology that allows Sigfox enabled devices to send small packets of data to the Sigfox Cloud (dedicated Sigfox servers on the internet). Sigfox uses a very light-weight protocol, so devices consume very little power and can operate over large distances.

As shown below, the Sigfox network consists of objects (devices), Sigfox Stations (gateways) and the Sigfox cloud (internet). Objects are Sigfox compliant sensors that send data over the air to the Sigfox base stations, which then routes the data to the Sigfox Cloud (also known as the Sigfox backend). The Adroit Sigfox Driver then communicates with the Sigfox cloud via an API or a call-back, to retrieve the objects data.

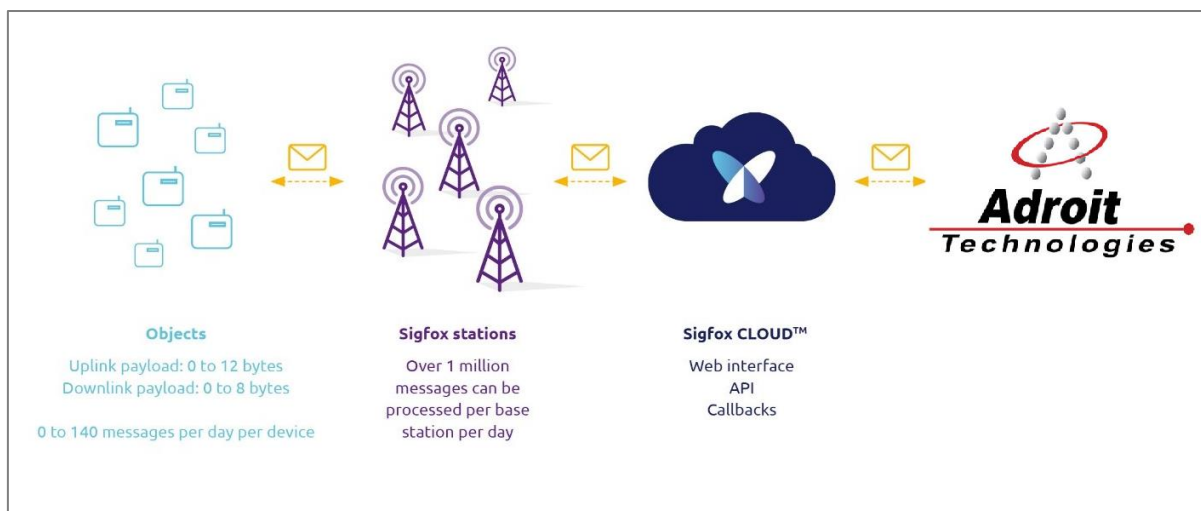


Figure 1 Sigfox Overview

- 1.
2. This Sigfox Driver document is divided into two parts:
 - Setting up and registering objects/devices onto the Sigfox cloud.
 - Setting up the Adroit Sigfox driver to communicate to the Sigfox cloud.



PLEASE NOTE

The Sigfox cloud can store a maximum of 1 years' worth of data per device.

2. Wiring

Not applicable.

3. Device Configuration

3.1. Part 1

3.1.1. Setting up Devices on the Sigfox Cloud

Before you can register your Sigfox devices onto the Sigfox Cloud, there are two things you must do:

Make sure there is Sigfox Coverage in the area that you are deploying the devices. If there is no Sigfox coverage, please contact South Africa's Sigfox Local Operator - Sqwidnet at <https://www.sqwidnet.com/>.

1. Buy a Sigfox contract and load tokens onto that contract. Each Sigfox device you are registering requires one token. There are three different types of contracts: 140 messages per day, 50 messages per day and 2 messages per day. If your device can send 140 messages per day, then go with this contract. Similarly, if your device can only send 2 messages per day, then go for the 2 messages/day contract. Contact your country's Sigfox operator for more information on the different types of contracts and how to purchase them. In South Africa, please contact South Africa's Sigfox Local Operator - Sqwidnet at <https://www.sqwidnet.com/>.
- 2.



PLEASE NOTE

If you need to register 10 devices onto a 140 messages/day contract, then you need 10 tokens on that 140 messages/day contract.



PLEASE NOTE

Each device type is associated to a specific contract, so make sure it is the correct contract (140/50/2 messages per day).

3.1.2. Sigfox Backend Hierarchal Structure

Figure 2 displays the hierarchal structure of the Sigfox backend, which is arranged in the following way:

1. **Groups:** This is the highest level. For example we have an Adroit group.
2. **Sub-groups:** Groups are divided into several sub-groups; we divide each customer into their own sub-group.
3. **Device types:** Group of devices with the same behavior. All sensors that have same manufacturer, payload, contract and call-back will fall into the same Device Type, for example Device_Type_Visio_Pressure.
4. **Devices:** Also known as objects, fall under a specific Device type category. They have a unique PAC ID, ID and a Product Certificate. For example, device_VisioPressure_123456.

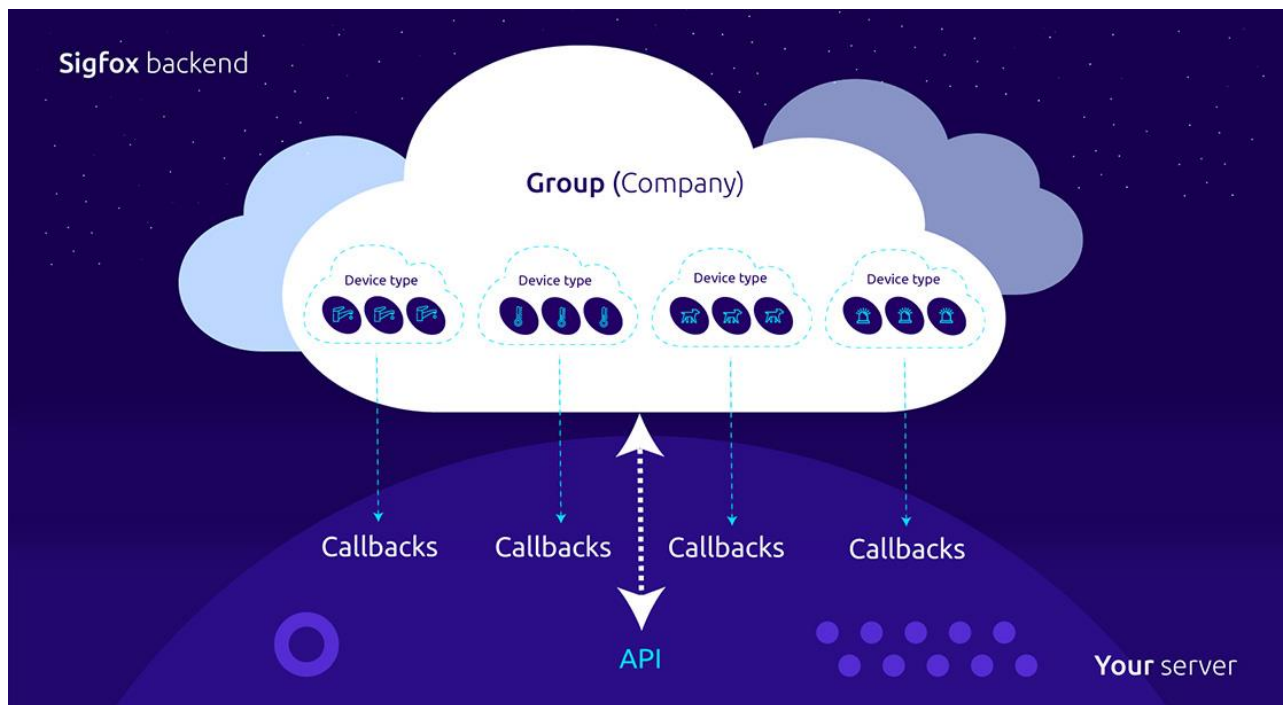


Figure 2 Sigfox Cloud Architecture

To register a new Sigfox device, we must choose a group, sub-group and device type which is associated with that device. As shown in Figure 3, choose the group, which the new device should belong too.

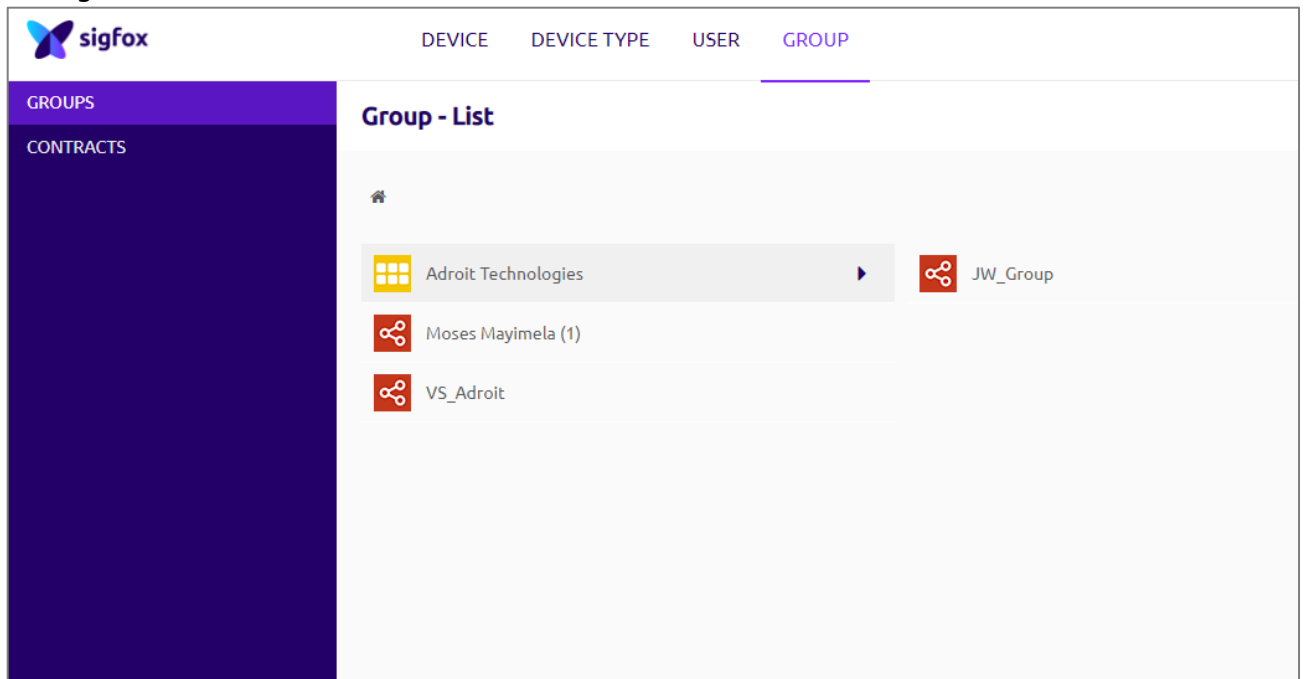


Figure 3: Sigfox Backend Groups and Sub-Groups

In figure 3, Adroit Technologies is the main group and JW_Group is a sub-group of the Adroit Technologies group. Use the sub-groups to divide customers of Adroit. For instance, if we have a new client called Tim, then create a Tim sub-group and connect all device types, contracts and devices associated to Tim on the Tim Sub-group. Keep in mind we can also give certain Sigfox users access to certain Sigfox groups and sub-groups.

3.1.3. Creating a new Device Type

Next, we need to create a new device type for our new device we want to register. Figure 4 shows a description of each parameter.

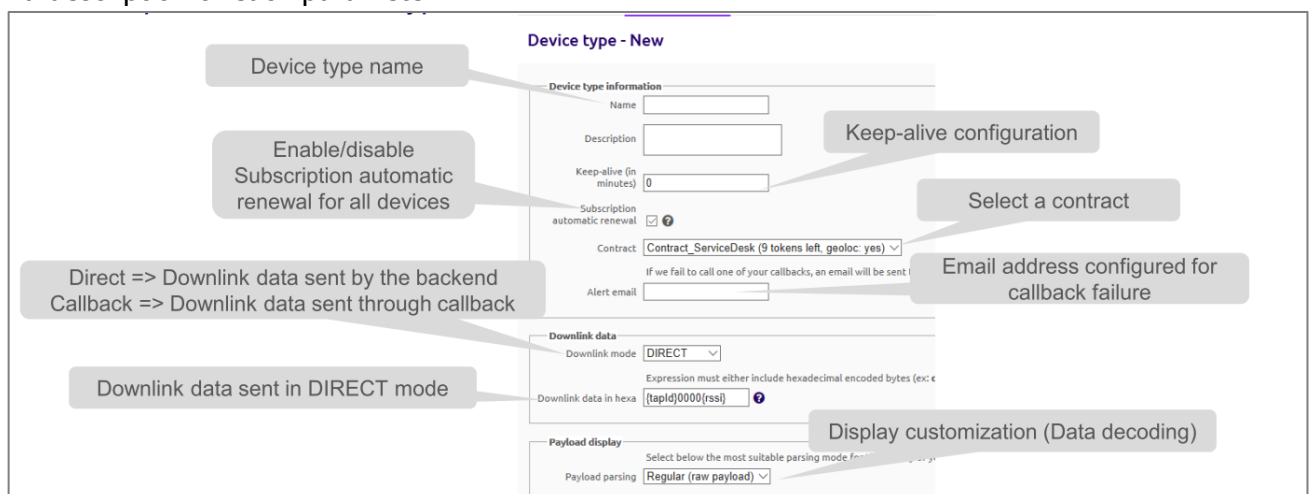


Figure 4: Creating a new Device Type



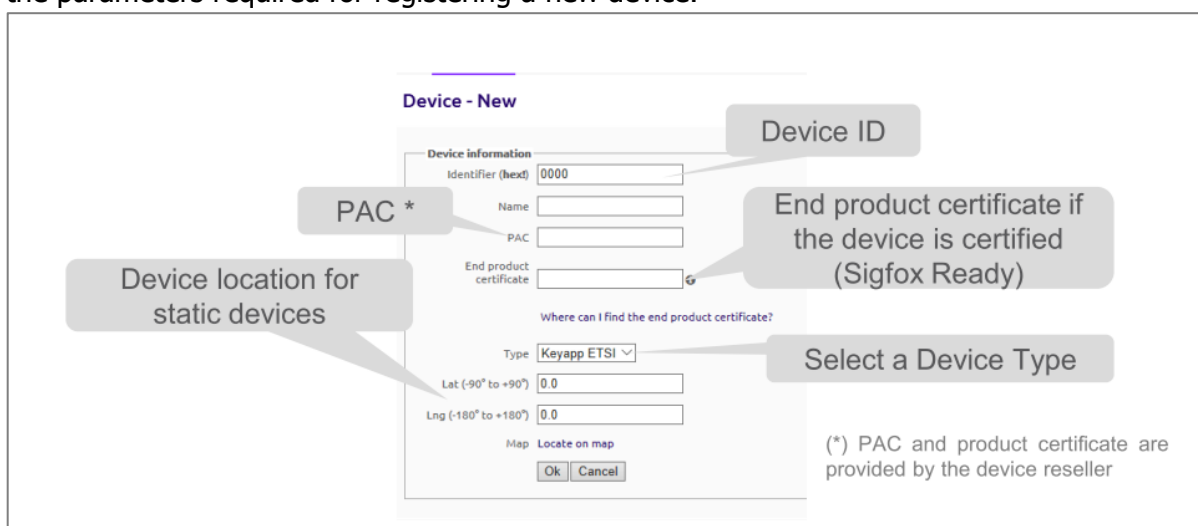
PLEASE NOTE

Contracts cannot be changed once device type is created.

In figure 4, the most important fields are the Device Type name (keep to naming conventions), the contract (discussed above), the payload parsing (keep this as Regular). Downlink data is if you are writing information down to the device from the Sigfox backend. This is not currently implemented in the Sigfox driver, so keep this field (Downlink Mode) as NONE.

3.1.4. Creating a new Device

Once the device type is created, then we can register our device to that device type. Figure 5 displays the parameters required for registering a new device.



The screenshot shows the 'Device - New' form with the following fields and callouts:

- Device ID:** Identifier (hex) field with value '0000'.
- PAC *:** PAC field.
- End product certificate if the device is certified (Sigfox Ready):** End product certificate field.
- Device location for static devices:** Lat (-90° to +90°) and Lng (-180° to +180°) fields.
- Select a Device Type:** Type dropdown menu with 'Keyapp ETSI' selected.

Additional text on the form includes: 'Where can I find the end product certificate?', 'Map Locate on map', and a note: '(*) PAC and product certificate are provided by the device reseller'.

Figure 5: Creating a New Device

In Figure 5, the Device ID (keep to naming conventions), PAC and end product certificate must be supplied by the manufacturer of the Sigfox device. For instance, if VisioSoft manufactured the device, then communicate with them to get this information.

3.1.5. Setting up the Sigfox Callback

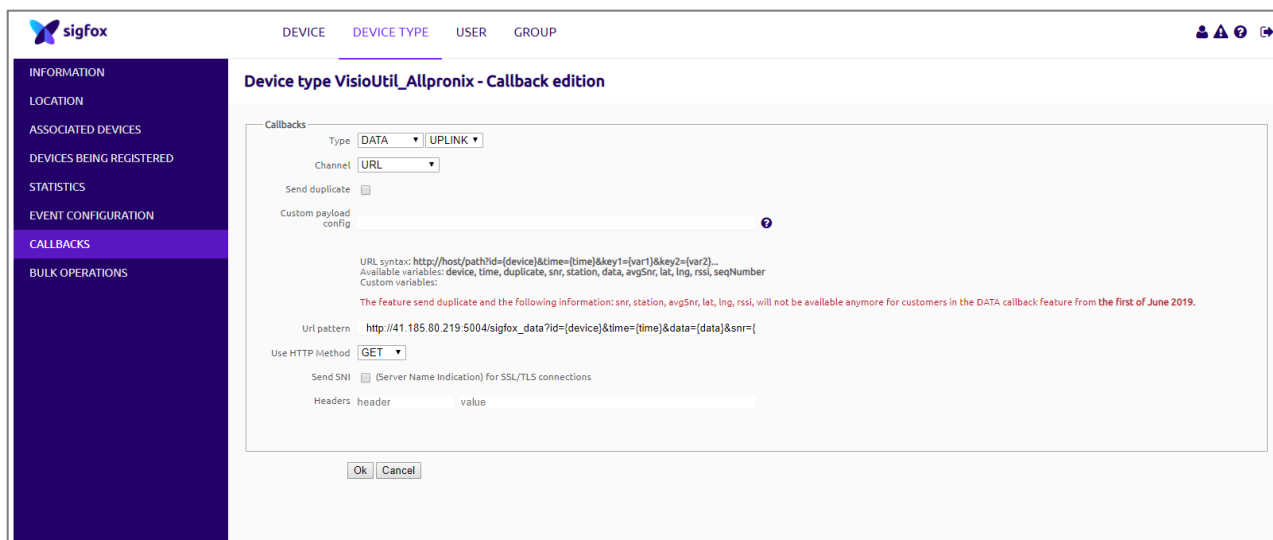
The Sigfox Callback is used to send data to a specific computer on the internet when the value of a device changes on the Sigfox backend. As shown in Figure 6, the callback is a query string with a public facing IP address (41.185.80.219) and an open port (5004) on the machine. This should be the IP address of the computer where Adroit is running. An example of the query string is:

http://41.185.80.219:5004/sigfox_data?id={device}&time={time}&data={data}&snr={snr}&station={station}&avgsnr={avgSnr}&lat={lat}&lng={lng}&rssi={rssi}&seqNumber={seqNumber}



PLEASE NOTE

Your computer must have a public IP address and the port in the callback string must be opened on the firewall on the computer that is running Adroit.



Device type VisioUtil_Allpronix - Callback edition

Callbacks

Type: DATA UPLINK

Channel: URL

Send duplicate: ☐

Custom payload config:

URL syntax: `http://host/path?id={device}&time={time}&key1={var1}&key2={var2}...`
 Available variables: device, time, duplicate, snr, station, data, avgSnr, lat, lng, rssi, seqNumber
 Custom variables:

The feature send duplicate and the following information: snr, station, avgSnr, lat, lng, rssi, will not be available anymore for customers in the DATA callback feature from the first of June 2019.

Url pattern: `http://41.185.80.219:5004/sigfox_data?id={device}&time={time}&data={data}&snr={`

Use HTTP Method: GET

Send SNI: ☐ (Server Name Indication) for SSL/TLS connections

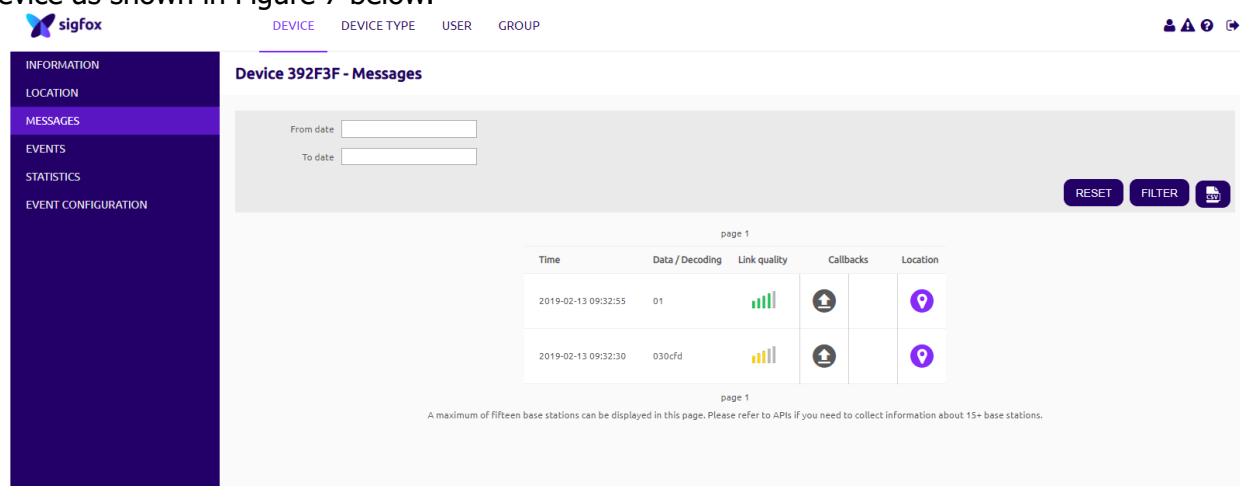
Headers: header value

Ok Cancel

Figure 6: Setting up a Sigfox Callback

3.1.6. Setup Validation

Now if you power up your Sigfox device, the Sigfox backend should receive messages from the device as shown in Figure 7 below.



Device 392F3F - Messages

From date:

To date:

RESET FILTER PDF

page 1

Time	Data / Decoding	Link quality	Callbacks	Location
2019-02-13 09:32:55	01			
2019-02-13 09:32:30	030cfd			

page 1

A maximum of fifteen base stations can be displayed in this page. Please refer to APIs if you need to collect information about 15+ base stations.

Figure 7: Setup Verification

3.2. Part 2

3.2.1. Setting Up the Adroit Sigfox Driver

At this point, your Sigfox enabled devices are registered on the Sigfox backend. The next step is to retrieve the devices information from the backend and display the data in Adroit.

Two important components are required for Adroit to understand a device type payload.

1. Firstly, a JavaScript file needs to be written and loaded into the Adroit Sigfox Driver, as shown in Figure 11. An example of this script file is shown in Figure 8:

```
function ProcessData(input)
{
    var Message_Flag_hex = input.substr(0,2);
    var R1_Min_hex = input.substr(2,3);
    var R1_Max_hex = input.substr(5,3);
    var R1_Mean_hex = input.substr(8,3);
    var R2_Min_hex = input.substr(11,3);
    var R2_Max_hex = input.substr(14,3);
    var R2_Mean_hex = input.substr(17,3);
    var Voltage_Reading_hex = input.substr(20,4);

    var Message_Flag = hexToInt(Message_Flag_hex) ;
    var R1_Min = hexToInt(R1_Min_hex) ;
    var R1_Max = hexToInt(R1_Max_hex);
    var R1_Mean = hexToInt(R1_Mean_hex) ;
    var R2_Min = hexToInt(R2_Min_hex) ;
    var R2_Max = hexToInt(R2_Max_hex) ;
    var R2_Mean = hexToInt(R2_Mean_hex) ;
    var Voltage_Reading = (hexToInt(Voltage_Reading_hex)/1000);

    var response="";
    response += "<Slot Name=\"Message Flag\">"+Message_Flag+"</Slot>";
    response += "<Slot Name=\"R1 Min\">"+R1_Min+"</Slot>";
    response += "<Slot Name=\"R1 Max\">"+R1_Max+"</Slot>";
    response += "<Slot Name=\"R1 Mean\">"+R1_Mean+"</Slot>";
    response += "<Slot Name=\"R2 Min\">"+R2_Min+"</Slot>";
    response += "<Slot Name=\"R2 Max\">"+R2_Max+"</Slot>";
    response += "<Slot Name=\"R2 Mean\">"+R2_Mean+"</Slot>";
    response += "<Slot Name=\"Voltage Reading\">"+Voltage_Reading+"</Slot>";

    return response;
}
```

Figure 8: Example of JavaScript File for Device Type

Please phone Adroit support if you are unsure how to write a new script for the Sigfox device. The script needs to parse the payload and put the relevant information into the correct slots. These slots are displayed at the bottom of Figure 8.

2. Secondly, an XML file with all relevant slots of that device type needs to be loaded into that IOT Agent. The slots are customizable for each instance of an IOT Agent. Figure 9 below shows an example of how the slots should be defined. Please load this XML file into your IoT Agent.

```
<IOT Name="Sigfox" Description="Sigfox Device">
  <Slots>
    <Slot Name="avgsnr" Description="Average Signal to noise ratio" Type="Real">11.11</Slot>
    <Slot Name="data" Description="Data sent" Type="String">sample</Slot>
    <Slot Name="id" Description="Sigfox ID" Type="String">sample</Slot>
    <Slot Name="lat" Description="Latitude" Type="Real">11.11</Slot>
    <Slot Name="lng" Description="Longitude" Type="Real">11.11</Slot>
    <Slot Name="rssi" Description="RSSI" Type="Real">11.11</Slot>
    <Slot Name="seqNumber" Description="Sequence Number" Type="Integer">1</Slot>
    <Slot Name="snr" Description="Latitude" Type="Real">11.11</Slot>
    <Slot Name="station" Description="Station ID" Type="String">sample</Slot>
    <Slot Name="time" Description="Timestamp" Type="String">1501850603</Slot>
    <Slot Name="local_time" Description="Local Timestamp" Type="String">1501850603</Slot>
    <Slot Name="alert" Description="for raising an alert" Type="Boolean">0</Slot>

    <Slot Name="Message_Flag" Message Flag Status Bits" Type="Real">0</Slot>
    <Slot Name="R1_Min" Description="R1 Min data" Type="Real">0</Slot>
    <Slot Name="R1_Max" Description="R1 Max data" Type="Real">0</Slot>
    <Slot Name="R1_Mean" Description="R1 Mean data" Type="Real">0</Slot>
    <Slot Name="R2_Min" Description="R2 Min data" Type="Real">0</Slot>
    <Slot Name="R2_Max" Description="R2 Max data" Type="Real">0</Slot>
    <Slot Name="R2_Mean" Description="R2 Mean data" Type="Real">1</Slot>
    <Slot Name="Voltage_Reading" Description="Voltage Reading" Type="Real">0</Slot>

  </Slots>
  <HeaderSlots>
    <Slot Name="Pressure_alert"/>
  </HeaderSlots>
</IOT>
```

Figure 9: Example of XML File for Device Type



PLEASE NOTE

These slots in Figure 8 must match the names (case sensitive) of the slot names of the XML file in Figure 9.

Once the Script and the XML files have been written. Then load the Script file into the Sigfox Driver Device configuration (Figure 11). And load the XML file into the Adroit Agent – Figure 10. Figure 10 displays the IOT Agent once the XML file has been loaded in.

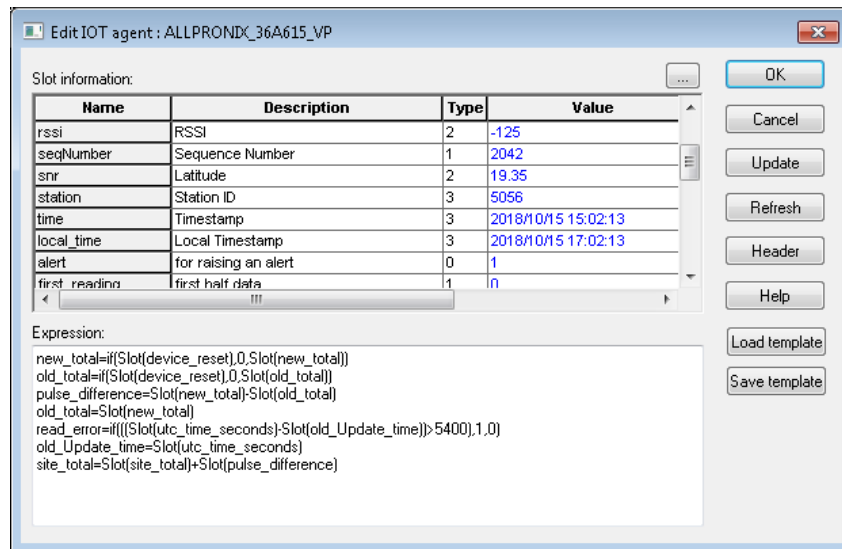


Figure 10: IOT Agent with XML File Loaded

In summary, if the new Sigfox Device has a payload of AABBCDD, where AA is two bytes for temperature, BB is two bytes for pressure, CC is two bytes for flow rate and DD are two bytes for the status of the device. Then the JavaScript file needs to parse the "AABBCDD" payload and then put the relevant bytes to the relevant XML slots. This JavaScript file then needs to be loaded in the Sigfox device configuration, and the XML file needs to be loaded into the IoT Agent.

3.3. Configuration Dialog

Setting up the Sigfox Driver is shown below:

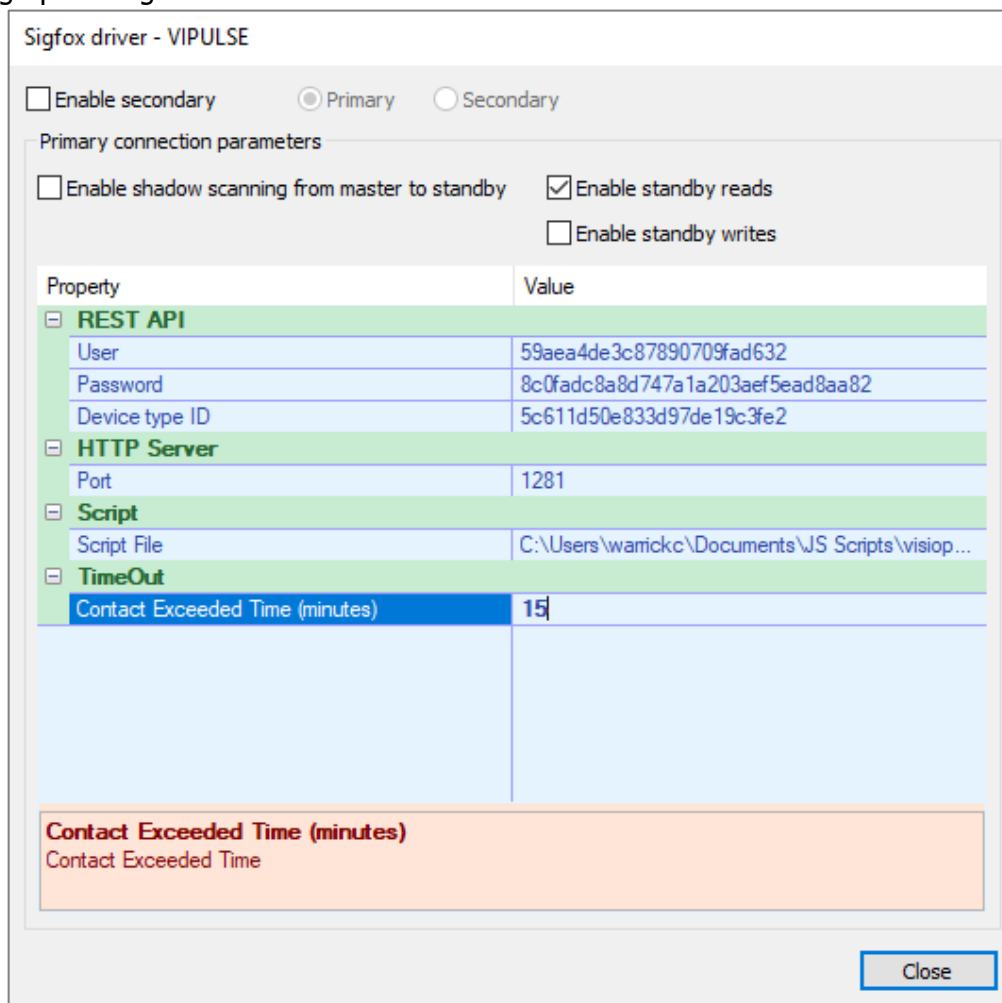


Figure 11: Sigfox Device Configuration

3.3.1. Driver Details

Enable Secondary – (Default = OFF) Allows enabling of redundant Sigfox Cloud connections such that in case of a failure, a switchover will occur.

Primary – Allows configuration setting of the Primary Sigfox Cloud connection.

Secondary – Allows configuration setting of the Secondary Sigfox Cloud connection.

3.3.2. Primary Connection Parameters

Enable shadow scanning from master to standby – (Default = OFF) If enabled will stop any communication to the Sigfox Cloud when the Agent server is in standby mode. By default, parallel scanning is assumed.

Enable standby reads – (Default = ON) Allows reading from the Sigfox Cloud when Agent Server is in standby mode.

Enable standby writes – (Default = OFF) Allows writing to the Sigfox Cloud when Agent Server is in standby mode.

3.3.3. REST API

User – This is the API Access Login name from the Sigfox Backend, see [Figure 13](#).

Password – This is the API Access Password from the Sigfox Backend, see [Figure 13](#).

Device type ID – The Device Type ID from the Sigfox Backend, as shown in [Figure 5](#).

3.3.4. HTTP Server

Port – The port which the Sigfox Backend will use to connect to Adroit. Make sure this port is open on the firewall and accessible from the internet. This port must be used in the Callback query string, as shown in [Figure 6](#).

3.3.5. Script

Script File – Load the script file to parse the Device Type payload.

3.3.6. TimeOut

Contact Exceeded Time – If Adroit does not receive a payload from a specific device within this amount of time, the device will mark bad. Due to the 140 message constraint, the minimum contact exceeded time is 10 minutes. A value of 0 will disable this functionality.

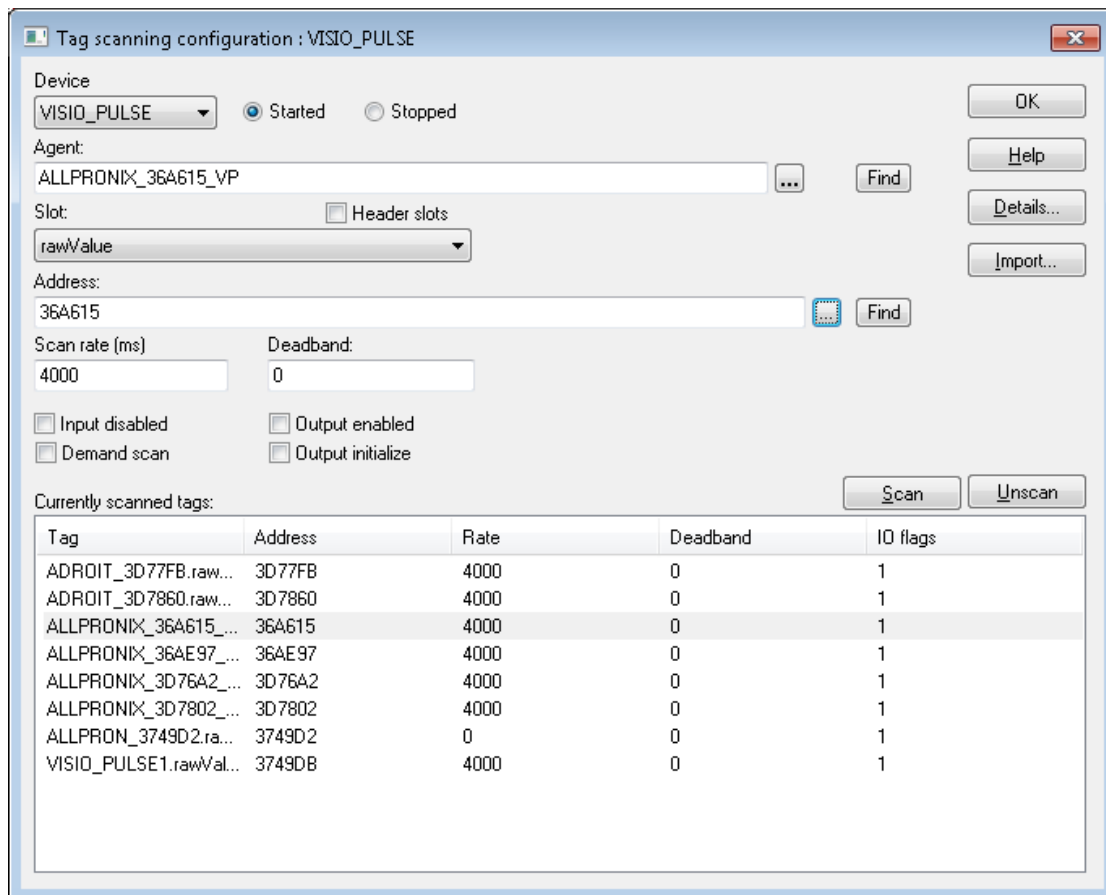


PLEASE NOTE

The computer that has Adroit installed and running must have internet access. In addition, all relevant Sigfox ports must be accessible from the internet.

4. Supported Addresses

The device ID is the scan address for the IoT Agent instance. Every device on the Sigfox network has its own ID, this is used as the scan address for the IoT Agent.



Tag scanning configuration : VISIO_PULSE

Device: **VISIO_PULSE** ☒ Started ☐ Stopped

Agent: ALLPRONIX_36A615_VP

Slot: ☐ Header slots
rawValue

Address: 36A615

Scan rate (ms): 4000 Deadband: 0

☐ Input disabled ☐ Output enabled
☐ Demand scan ☐ Output initialize

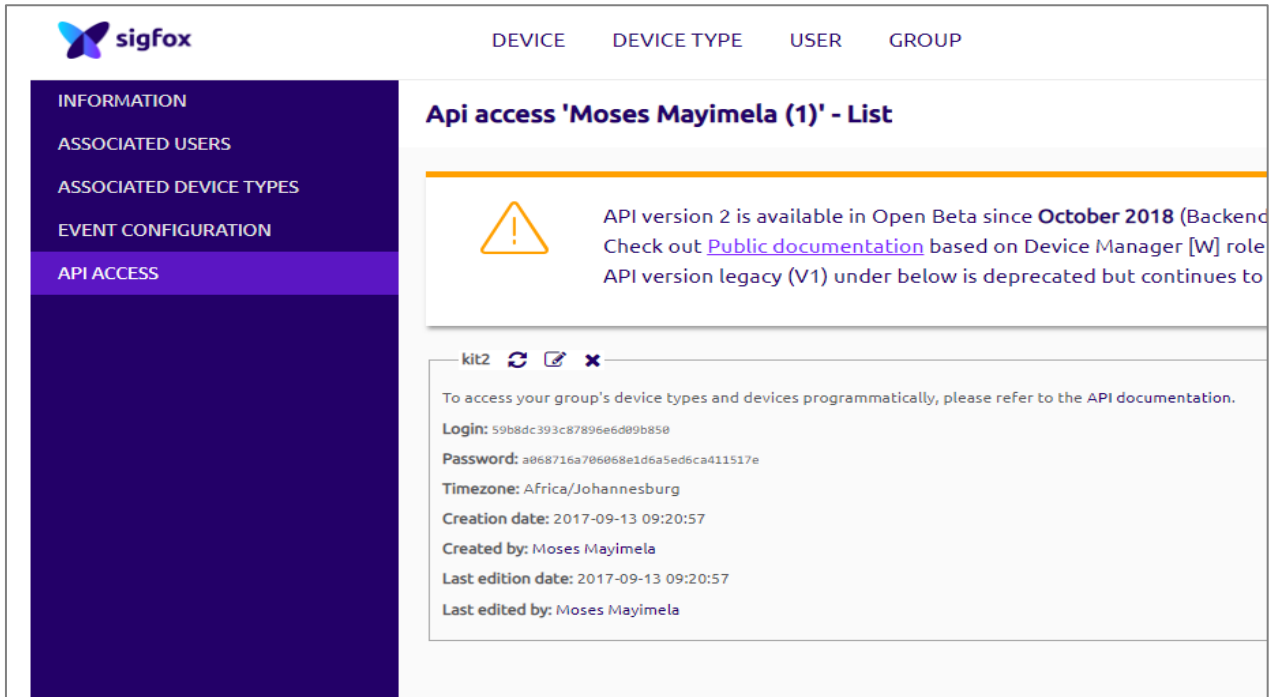
Currently scanned tags:

Tag	Address	Rate	Deadband	IO flags
ADROIT_3D77FB.raw...	3D77FB	4000	0	1
ADROIT_3D7860.raw...	3D7860	4000	0	1
ALLPRONIX_36A615_...	36A615	4000	0	1
ALLPRONIX_36AE97_...	36AE97	4000	0	1
ALLPRONIX_3D76A2_...	3D76A2	4000	0	1
ALLPRONIX_3D7802_...	3D7802	4000	0	1
ALLPRONIX_3749D2.ra...	3749D2	0	0	1
VISIO_PULSE1.rawVal...	3749DB	4000	0	1

Figure 12: Scanning Device ID to Sigfox Device Address

4.1. Example

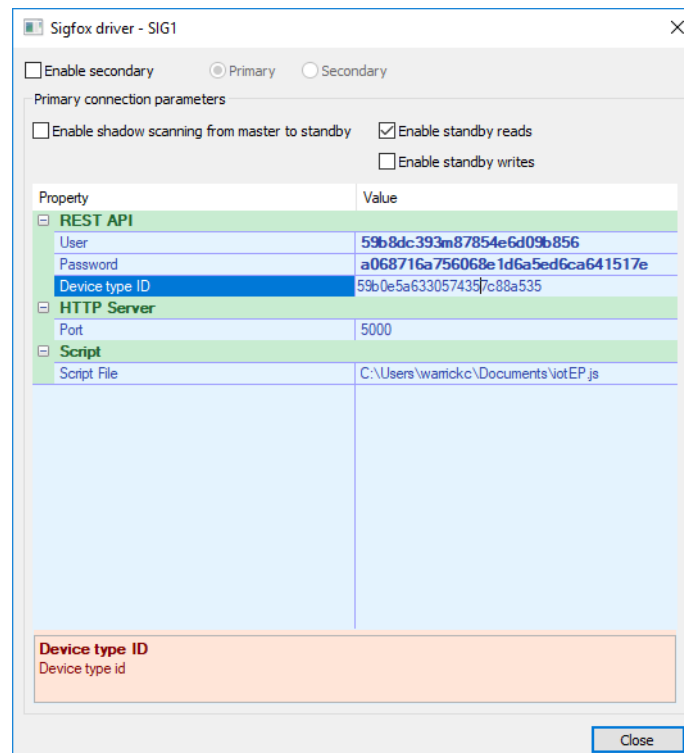
Every group (discussed above), will have its own API ACCESS as shown in Figure 13. Figure 13 shows how to get the User name and password for your Sigfox group. This information is needed in Figure 14 below.



The screenshot shows the Sigfox web interface. On the left is a navigation menu with options: INFORMATION, ASSOCIATED USERS, ASSOCIATED DEVICE TYPES, EVENT CONFIGURATION, and API ACCESS (which is highlighted). The main content area is titled 'Api access 'Moses Mayimela (1)' - List'. It contains a warning message about API version 2 being available in Open Beta since October 2018, and a note that API version legacy (V1) is deprecated but continues to be supported. Below this, there is a section for 'kit2' with a list of API access details:

- To access your group's device types and devices programmatically, please refer to the [API documentation](#).
- Login:** 59b8dc393c87896e6d09b856
- Password:** a068716a706068e1d6a5ed6ca411517e
- Timezone:** Africa/Johannesburg
- Creation date:** 2017-09-13 09:20:57
- Created by:** Moses Mayimela
- Last edition date:** 2017-09-13 09:20:57
- Last edited by:** Moses Mayimela

Figure 13: API Access Information



The screenshot shows the 'Sigfox driver - SIG1' window. It has a tabbed interface with 'Primary' selected. The 'Primary connection parameters' section contains several checkboxes: 'Enable secondary' (unchecked), 'Enable shadow scanning from master to standby' (unchecked), 'Enable standby reads' (checked), and 'Enable standby writes' (unchecked). Below this is a table with properties and values:

Property	Value
REST API	
User	59b8dc393c87896e6d09b856
Password	a068716a706068e1d6a5ed6ca411517e
Device type ID	59b0e5a6330574357c88a535
HTTP Server	
Port	5000
Script	
Script File	C:\Users\warrickc\Documents\IoT EP.js

At the bottom of the window, there is a section for 'Device type ID' with a text box containing the value 'Device type id'. A 'Close' button is located at the bottom right.

Figure 14: Sigfox Driver Setup Example

5. Protocol Definition and Considerations

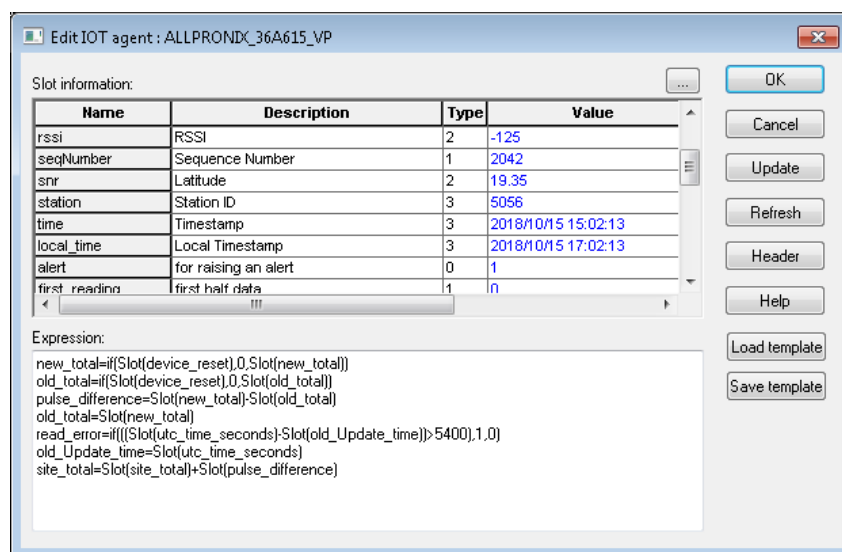
5.1. Final Verification

Once this has all been completed. Power your Sigfox device and check the backend for incoming messages. Check the date and time to make sure the newest messages are coming in. If there are no new messages, then check your Sigfox network coverage, Sigfox device battery life and your contract (1/50/140 messages per day).

Time	Data / Decoding	LQI	Callbacks	Location
2019-02-26 22:51:12	c000000006de01000902c599			
2019-02-25 22:51:12	c0000000ffda0100990034f1			
2019-02-24 22:51:12	c0000000f2da01007600fbbf			
2019-02-23 22:51:13	c0000000dada01003e029623			
2019-02-22 22:51:13	c00000004dd80100c3010e34			

Figure 15: Sigfox Backend Messages

If the latest messages are coming into the Sigfox backend, then check that your Adroit Agents slots are being updated with the newest values.



Edit IOT agent : ALLPRONDX_36A615_VP

Slot information:

Name	Description	Type	Value
rsssi	RSSI	2	-125
seqNumber	Sequence Number	1	2042
snr	Latitude	2	19.35
station	Station ID	3	5056
time	Timestamp	3	2018/10/15 15:02:13
local_time	Local Timestamp	3	2018/10/15 17:02:13
alert	for raising an alert	0	1
first_reading	first half data	1	0

Expression:

```

new_total=iif(Slot(device_reset),0,Slot(new_total))
old_total=iif(Slot(device_reset),0,Slot(old_total))
pulse_difference=Slot(new_total)-Slot(old_total)
old_total=Slot(new_total)
read_error=iif((Slot(utc_time_seconds)-Slot(old_Update_time))>5400,1,0)
old_Update_time=Slot(utc_time_seconds)
site_total=Slot(site_total)+Slot(pulse_difference)
  
```

Buttons: OK, Cancel, Update, Refresh, Header, Help, Load template, Save template

Figure 16: IOT Agent Slots

If you are not getting the latest values into your Agent slots, then check the JavaScript file, the XML file, the Device ID that is scanned. The relevant ports are opened and if you have internet access to that machine.

6. General Notes and Troubleshooting

6.1. Contact Details

Email: support@adroit.co.za and drivers@adroit.co.za

Phone: +27 11 658-8100

Web: www.adroit.co.za

6.2. Adroit Technologies Knowledge Base

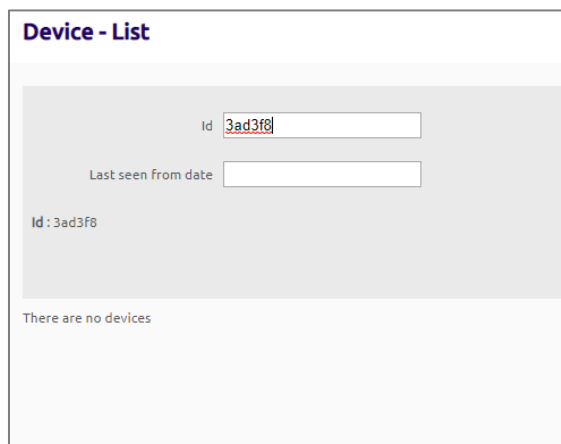
For additional information, please refer to our Knowledgebase website:

[Latest Drivers topics - Features, discussions, tips, tricks, questions, problems and feedback \(adroittechnologiesautomation.com\)](http://adroittechnologiesautomation.com)

6.3. Troubleshooting

When troubleshooting a Sigfox Driver problem, it is important to understand where the source of the problem is. Please use the Sigfox device Datascope as a tool to try debug the problem. The following steps will hopefully be a guide in troubleshooting a faulty Sigfox device that is trying to communicate to Adroit.

1. Step 1: As shown in Figure 17 below, if the device cannot be found on your Sigfox backend, then you need to register that device onto the Sigfox backend (see [Part 1](#) of this document). If it was registered, then check if it was deleted, moved or if you are searching in the correct Sigfox group.



The screenshot shows a web interface titled "Device - List". It contains a search area with two input fields: "Id" with the value "3ad3f8" and "Last seen from date". Below these fields, the text "Id : 3ad3f8" is displayed. At the bottom of the interface, a message states "There are no devices".

Figure 17: Device Not Found

2. Step 2 is to check if the device and device type contracts have expired. In Figure 18 below, the `adroit_t_50bb_820c` contract is clickable, this is where you can check if the devices and device type contracts are still valid.

Communication status: ●
Contract: adroit_t_50bb_820c
Activation date: 2019-03-01 18:07:55
Token validity: 2020-03-01
Unsubscription date: N/A Set ?
Subscription automatic renewal status: Not allowed !
Subscription automatic renewal: ☒ ?
Creation date: 2019-03-01 15:26:20

Figure 18: Check Contract Validity

- Step 3 is to check if the information on the Sigfox backend is correct for that specific device, as shown in Figure 19 below.

page 1

Time	Data / Decoding	LQI	Callbacks	Location
2019-07-30 12:55:49	00e40e40e40e40e41e400cc1			
2019-07-30 12:40:50	00e3fe40e3fe40e41e400cd5			
2019-07-30 12:25:53	00e3fe40e3fe3fe40e3f0ccb			
2019-07-30 12:10:53	00e3fe40e3fe3fe40e3f0cd5			

Figure 19: Verify Device Payloads

- If the signal strength is bad, then packets could be lost. Check the times of each message and check if this correlates to the devices contract (140/50/2 messages per day). Also check if the payload of these messages are what you expect, if you are unsure how to interpret the payload, then contact Adroit support or the device manufacturer. This is the first place to check if you are reading incorrect values in Adroit from the device.
 - If the timestamp is very old, and a lot of messages are being lost, then check if the device is turned off, if the battery died or if the device moved to a location with bad signal strength.
- If the callbacks are failing, as shown in Figure 20 below. Then the callback URL for the device type associated to that device might be incorrect. The port needs to be opened on the relevant firewalls and servers and the IP address must be a static public IP address. If the Agent Server is not running on the machine waiting for the callback, then the callback will also fail. Note that multiple callbacks can be setup for a device type and if one of them fails (and the rest succeed), then the icon will go red. Also make sure the User, Password and Device Type ID are correct in the Adroit Sigfox Driver Configuration.



Figure 20: Callback Failure

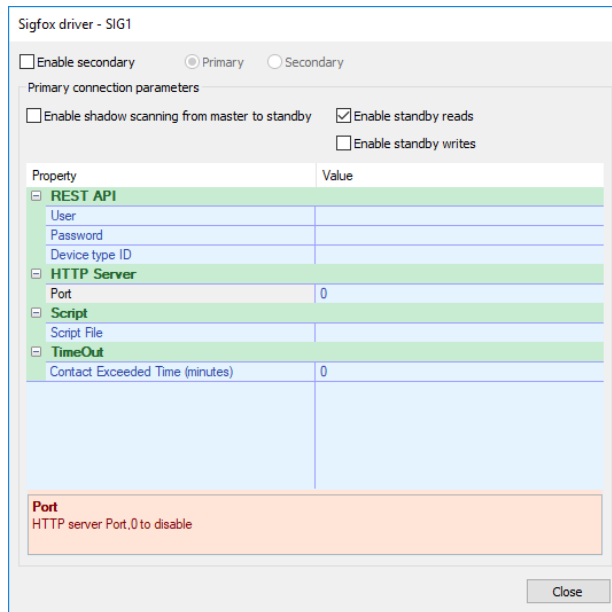
At this point, the devices callbacks should be successful, the messages should have healthy signal, the device/device type contracts are still valid and the payload of the device on the backend is all correct. If the information in Adroit is still incorrect then there is probably a problem with the Adroit Sigfox Driver Setup. This is discussed below:

5. As shown in Figure 21, every field in the Sigfox device configuration needs to be completed correctly. Refer to [Part 2](#) in this document.



PLEASE NOTE

The port in the device configuration must be unique per device type and cannot be shared across multiple device types.



Sigfox driver - SIG1

☐ Enable secondary ☒ Primary ☐ Secondary

Primary connection parameters

☐ Enable shadow scanning from master to standby ☒ Enable standby reads ☐ Enable standby writes

Property	Value
☒ REST API	
User	
Password	
Device type ID	
☒ HTTP Server	
Port	0
☒ Script	
Script File	
☒ TimeOut	
Contact Exceeded Time (minutes)	0

Port
HTTP server Port,0 to disable

Close

Figure 21: Sigfox Device Configuration

- If this Sigfox devices information in Adroit is still inaccurate, then please check the script file that parses the payload, the expression field of the IoT Agent and the XML file associated to that Sigfox Device. Refer to [Part 2](#) of this document.
- If the problem persists please contact Adroit Support for assistance.

7. Revisions

7.1. Document Revision

Rev.	Date	Author	Comment	Approval
1.0	14-Dec-2022	J. Duncan	New Format	D. Jordaan
1.1	18-Jan-2023	K. Robinson	General formatting and content editing.	D. Jordaan
2.0				
3.0				
4.0				
5.0				
6.0				

7.2. DLL Revision

Rev.	Date	Changes
1	20-Feb-2019	First Release of Driver.
	09-Jul-2019	Added contact exceeded timeout
20	17-Aug-2020	Using Ipworks.2016 libraries